AI tools, under control.

Model proposes. Application authorizes. Backend commits.

Spring AI 2.0.1 on **:8080** orchestrates; Java Hotel Service on **:9090** owns booking and payment state.

THE REQUEST PATH

Postman — Basic Auth → **ChatService** — IDs + tools → **OpenAI** — Name + JSON → **Spring AI** — @Tool dispatch → **integration** — Safe HTTP call → **Hotel Service** — Business state

Return loop: tool result goes back to OpenAI for the final answer. Actual reservation ID/status also appear in the structured **reservations[]** response.

01 Permissions are two gates

Offer only allowed tools. Then authorize the actual tool and integration-service entry.

reader	Search + reservation status
booker	Reads + create when key is valid
chat	Conversation only; no tools

SMALL CODE, STRONG BOUNDARY

```
var allowed = toolExposure.toolsFor(
    UserContext.current(), hasKey);
if (allowed.length > 0)
    prompt.tools(allowed);

@PreAuthorize(
  "hasAuthority('RESERVATION_CREATE')")
```

Roles come from **Spring Security**, never the prompt or X-Role. No shared default tools. Basic Auth passwords come from environment variables.

02 Protect three different scopes

LIMITS / QUANTITY

Spring AI: 3 tool calls total, 2 per tool, per chat request.

FINGERPRINT / DUPLICATE INTENT

Tool name + normalized arguments; reject repeated writes within this execution.

IDEMPOTENCY / BUSINESS EFFECT

Backend stable key prevents another booking on HTTP retries or replay.

```
@CircuitBreaker(name="availability")
@Retry(name="availability")
@TimeLimiter(name="availability")
CompletableFuture<AvailabilityResponse>
    search(...)
```

@Tool stays synchronous. The gateway uses a bounded executor + RestClient. Service maps final failure to a structured result.

03 Booking is a state machine

PENDING_PAYMENT — Hold inventory + outbox → **PAYMENT WORKER** — Stable charge key; outside DB tx → **CONFIRMED** — Confirmed event + notifications

Success: confirm + outbox. **Decline:** cancel + release inventory. **Unknown payment outcome:** hold and reconcile using the same key; do not blindly charge again.

Payment token arrives via **Payment-Method-Token**; operation key via **Idempotency-Key**. Never ask the LLM to collect card details or generate a payment token.

Email/PMS delivery is simulated with independent durable consumers. Real payment and SNS/SQS transport are not connected. **chat.success != reservation.CONFIRMED.**

SENIOR INSIGHT: One tool call may make 3 HTTP attempts. Resilience uses CircuitBreaker(Retry(TimeLimiter(HTTP))): up to 3 attempts, 2 seconds per attempt, 200/400 ms backoff. Tool caps do not create an overall deadline or token budget.

Prove it. Trace it. Explain it.

Execution tests prove the plumbing. Live evaluations measure model decisions.

04 Logs are a flight recorder; DB rows are receipts

```
[CHAT] user.message.received
[MODEL] requestedTools=[searchAvailability]
[TOOL] tool.started searchAvailability
[SERVICE] AvailabilityService.search
[EXTERNAL API] httpStatus=503 attemptNo=1
[EXTERNAL API] httpStatus=200 attemptNo=2
[TOOL] tool.call status=COMPLETED
[CHAT] turn.completed toolsCalled=[searchAvailability]
```

Who records what? AuditModelAdvisor observes model calls; ToolAudit observes tools; AuditHttpInterceptor observes HTTP attempts. **AuditStore** is their shared writer.

Console = short flow. **logs/ai-chat.jsonl** = detailed events. **reservation_audit** = important booking-write outcomes only. AI2 audit events omit raw sensitive payloads.

IDS: ONE SESSION, MANY REQUESTS

conversationId = C1
Groups related requests.

Search request
turnId U1 / traceId T1

Booking request
turnId U2 / traceId T2

Same trace across that request's tools and HTTP retries. New request = new turn + trace. **Conversation ID is not memory.**

05 Evaluate decisions, not side effects

Contract: prompt + schemas -> raw model -> captured calls -> assertions. Callbacks cannot execute.

```
record ToolEvalCase(
  String userPrompt, String expectedTool,
  Map<String,Object> expectedArguments,
  Set<String> forbiddenTools,
  boolean expectNoTool,
  boolean confirmationRequired) {}
```

Dates + guests -> searchAvailability(args)

"Thanks" / missing dates -> no tool

Unconfirmed / reader books -> no write

Check **every** call: forbidden tools, extra calls, exact arguments and valid JSON. 15 scenarios.

```
RUN_LIVE_TOOL_EVALS=true mvn \
  -Dtest=LiveToolSelectionEvalTest test
```

Live needs **OPENAI_API_KEY** (API usage). Report: **target/tool-evals/results.jsonl**. Provider 401 = authentication failure, not selection failure.

confirmationRequired is not a server-side consent gate.

06 Investigate the first broken edge

1. Correlate
Copy response.traceId; follow this request only.

2. Separate intent from execution
MODEL requestedTools is a proposal. TOOL tool.call proves invocation.

3. Locate the boundary
No HTTP attempt? Check validation, permissions, duplicate/limit rejection or open circuit.

4. Inspect each HTTP attempt
Status + attemptNo + latency + errorCode explain retries and timeouts.

5. Verify business truth
Read reservations[0].reservationId, then GET /api/reservations/{id}. Pending is not confirmed.

```
python3 scripts/read_trace.py \
  --trace-id <id> --event tool.call
```

/api/audit/events?traceId=... reads business rows, not full traces. Search-only requests can correctly return no DB events.

07 The honest production answer

Implemented: native call limits, dynamic permissions, duplicate-write detection, backend idempotency, async resilience, structured logs and one AI audit table. No RAG or persistent chat memory.

Next boundaries: real identity + ownership + server-side consent; total deadline/token budgets; OpenTelemetry; real payment/SNS/SQS; backend log redaction. Basic Auth and staff-wide reads are demo choices.

60-second recall: Who may act? Which tool should run? How is the call bounded? Can a retry duplicate a side effect? Which evidence proves the result? Then explain the exact layer enforcing each answer.